Notes for ECE 46800 - Introduction to Compilers and Translation Engineering

Zeke Ulrich

August 24, 2026

Contents

Course Description

The design and construction of compilers and other translators. Topics include compilation goals, organization of a translator, grammars and languages, symbol tables, lexical analysis, syntax analysis (parsing), error handling, intermediate and final code generation, assemblers, interpreters, and an introduction to optimization. Emphasis is on engineering a compiler or interpreter for a small programming language - typically a C or Pascal subset. Projects involved the stepwise implementation (and documentation) of such a system.

Compiler Basics

A *compiler* is commonly understood as a program that converts a high-level language to a low-level language executable by hardware. However, in general a compiler translates one representation of a program to another, and perhaps does some analysis along the way.

All translation is transformation. A Greek-to-English translation of the Odyssey will preserve the meaning while reading as close as possible to the original. Code translation is the same. There are many possible translations of a given program to another language, with different properties and optimizations. Two translations that accomplish the same task may run in different times or use different amounts of memory.

Compilers save poor programmers from writing directly in assembly. The many flavours of high-to-low compilers enable us to write in one language and compile to many different architectures, instead of needing to write the same program in x86 for x86 machines, RISC-V for RISC-V chips, and so on.